```
$ docker build -t railsapp .
Sending build context to Docker daemon  102.9kB
Step 1/6 : FROM ruby:2.6
 ---> f28a9e1d0449
Step 2/6 : RUN apt-get update -yqq && apt-get install -yqq --no-install-
recommends   nodejs
 ---> Using cache
 ---> 29c3dee2b8c7
Step 3/6 : COPY . /usr/src/app/
 ---> fff98079f6ac
Step 4/6 : WORKDIR /usr/src/app
 ---> Running in 3e36b19fecbf
Removing intermediate container 3e36b19fecbf
 ---> 34e46dae43ab
Step 5/6 : RUN bundle install
 ---> Running in f4528be7eb2b
«...»
Bundle complete! 15 Gemfile dependencies, 68 gems now installed.
Bundled gems are installed into `/usr/local/bundle`
«...»
Removing intermediate container f4528be7eb2b
 ---> 5965a3004093
Step 6/6 : CMD ["bin/rails", "s", "-b", "0.0.0.0"]
 ---> Running in fe59ed9392a7
Removing intermediate container fe59ed9392a7
 ---> 1fbb2af53579
Successfully built 1fbb2af53579
Successfully tagged railsapp:latest
```

Wow, that took quite a while. The biggest cause of this slowness was that all our gems were rebuilt from scratch, yet all we did was change our README.md file—what gives?

If you look through the output, you'll see that steps 1 to 3 all say Using cache. Docker didn't have to rebuild those layers because it could compare the Dockerfile instruction with the cached intermediate image for that step, and see that they were the same.

However, that's not the case with step 4 (COPY . /usr/src/app)—this step isn't using the cache. Although the Dockerfile instruction remained the same, since it's a COPY instruction, Docker checks the files being copied (excluding any in the .dockerignore file) and compares them with those copied previously. It realizes that README.md has changed, so it knows to rebuild from this step.

There's no getting around the fact that, if files change, a new image needs to be created containing the changed files. However, in our case, it's unfortunate that merely changing the README.md causes bundle install to be run again. It's both slow and completely unnecessary: our change to README.md had no impact